

c language

Control structure

c language

statement

- In C language, a *statement* is one or more lines of code ending with a semi-colon (;).
- It generally contains expressions (expressions have a value).

c language

statement

- **Simple Statement**

- Basic building blocks of a program
- One or more lines terminated by a semicolon (;)

```
printf("Hello, World!");
```

- **Compound Statement or Block**

- Used to organize simple statements into complex structures
- One or more statements enclosed by braces { }
- Recommended to indent blocks for readability

```
int main() {  
    printf("Hello, ");  
    printf("World!");  
}
```

Control **structure**

- **Control structures allow to change the ordering of how the statements in a program is executed**
- Types of control structures:
 - *Decision control structures*
 - allow to select specific sections of code to be executed
 - *Repetition control structures*
 - allow to execute specific sections of the code a number of times

Control **structure**

- C language statements that allows a programmer to select and execute **specific blocks of code** while skipping other sections.
- Include types such as:
 - **If**
 - **If-Else**
 - **If-Else-If**
 - **Switch structure**

if

structure

If Structure

- The *if* statement specifies that a statement (or block of code) will be executed **if and only if** a certain Boolean statement is **true**.

- The general format of an *if* statement is:

```
if (expression)  
    statement or block;
```

or

```
if (expression) statement;
```

where *expression* represents a relational, equality, or logical expression (***conditional expression***).

if structure

- If there is a need to execute several statements (***compound statement***), then left and right {} braces can group these statements.

```
if (expression)  
{  
    statement1;  
    statement2;  
    ...  
}
```

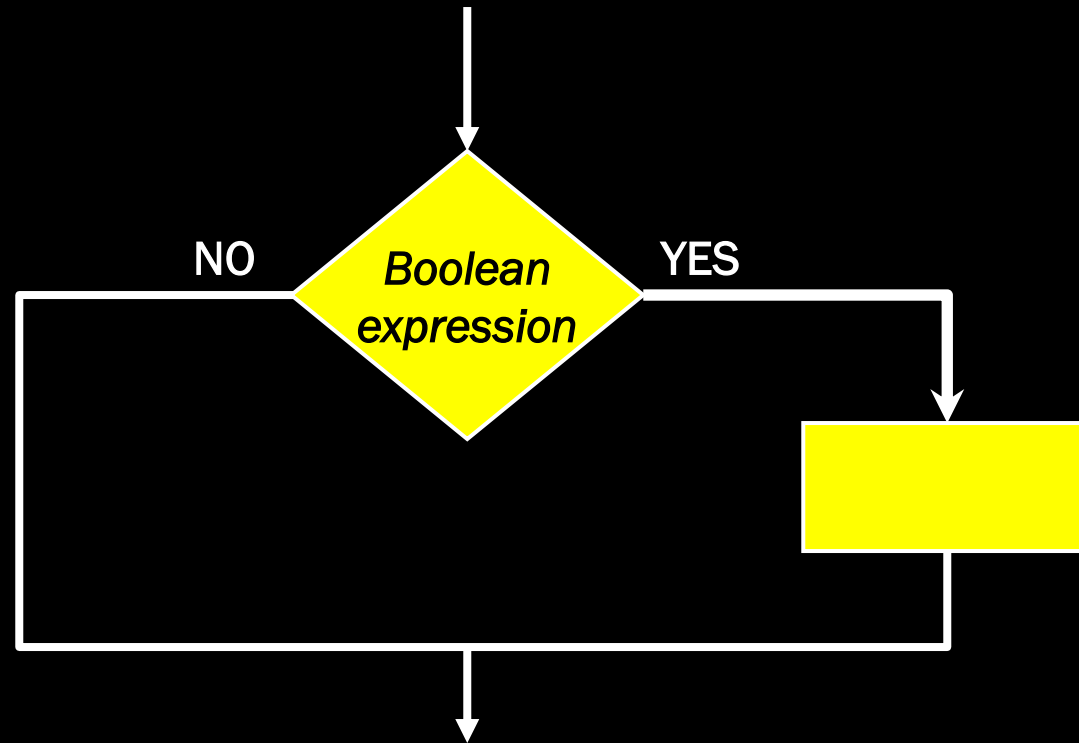
if structure

Every time C language encounters an *if* statement,

1. It determines whether *expression* is *true* or *false*.
2. If *expression* is *true*, then C language *executes statement*.
3. If *expression* is *false*, then C language *ignores* or *skips* the remainder of the *if* statement and proceeds to the next statement.

Control structure

- Making a decision involves choosing between two alternative courses of action based on some value within a program.



if structure

If Structure: Example 1

- Suppose that the passing grade on an examination is 75 (out of 100). Then the if statement may be written in Java as:

```
if (grade >= 75)  
    printf("You Passed!");
```

if

structure

If Structure: Example 2

```
int grade = 68;  
if (grade > 60)  
{  
    printf("Congratulations!");  
    printf("You Passed!");  
}
```

Exercises:

1. Test if the value of the variable count is greater than 10. If the answer is yes, print "Count is greater than 10".

if else structure

If-Else Structure

- The *if-else* statement is used when you want to execute one statement if a condition is true, and another statement if the condition is false.
- The general format of an *if-else* statement is:

```
if (expression)  
    statement1;  
else  
    statement2;
```

or

```
if (expression) statement1;  
else statement2;
```

if

else structure

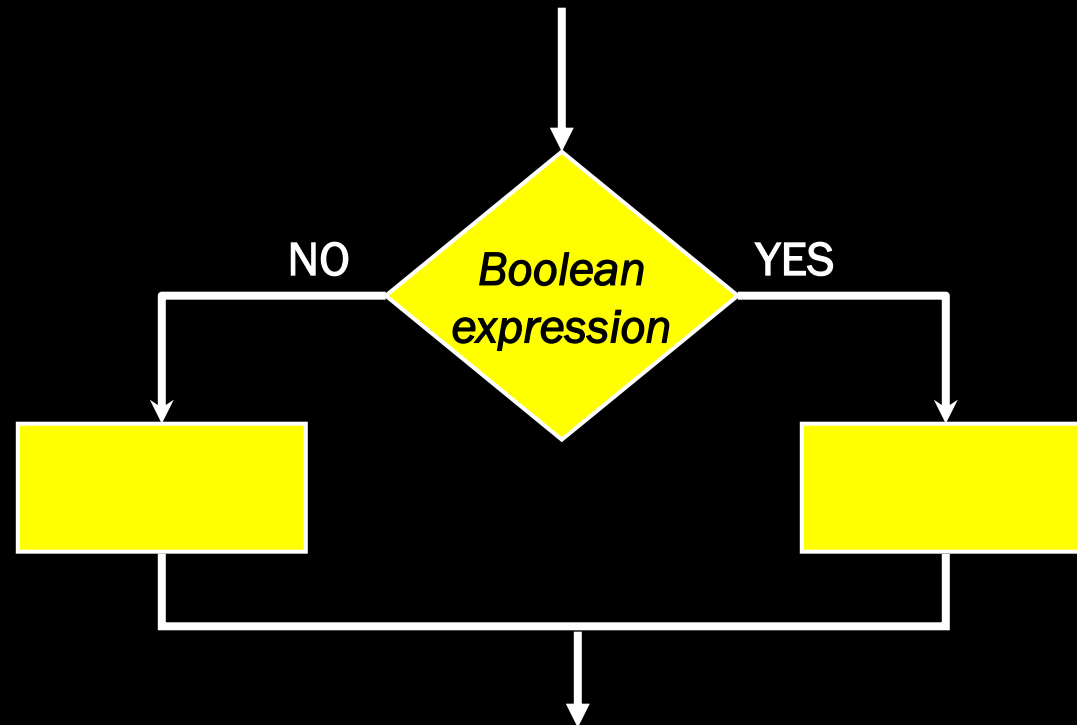
- As in the *if* statement, compound statements may also be used in *if-else* statements (provided they are grouped together by braces).

```
if (expression)
{
    statement1;
    statement2;
    ...
}
else
{
    statement3;
    statement4;
    ...
}
```

if

else structure

- Making a decision involves choosing between two alternative courses of action based on some value within a program.



if else structure

If-Else Structure: Example 1

```
int grade = 68;  
if (grade > 60)  
    printf("Congratulations!");  
else  
    printf("Sorry you failed!");
```


if else structure

If-Else Structure: Example 2

```
int grade = 68;  
if (grade > 60) {  
    printf("Congratulations!");  
    printf("You Passed!");  
}  
else {  
    printf("Sorry you failed!");  
}
```

if else structure

If-Else Structure: Example 3

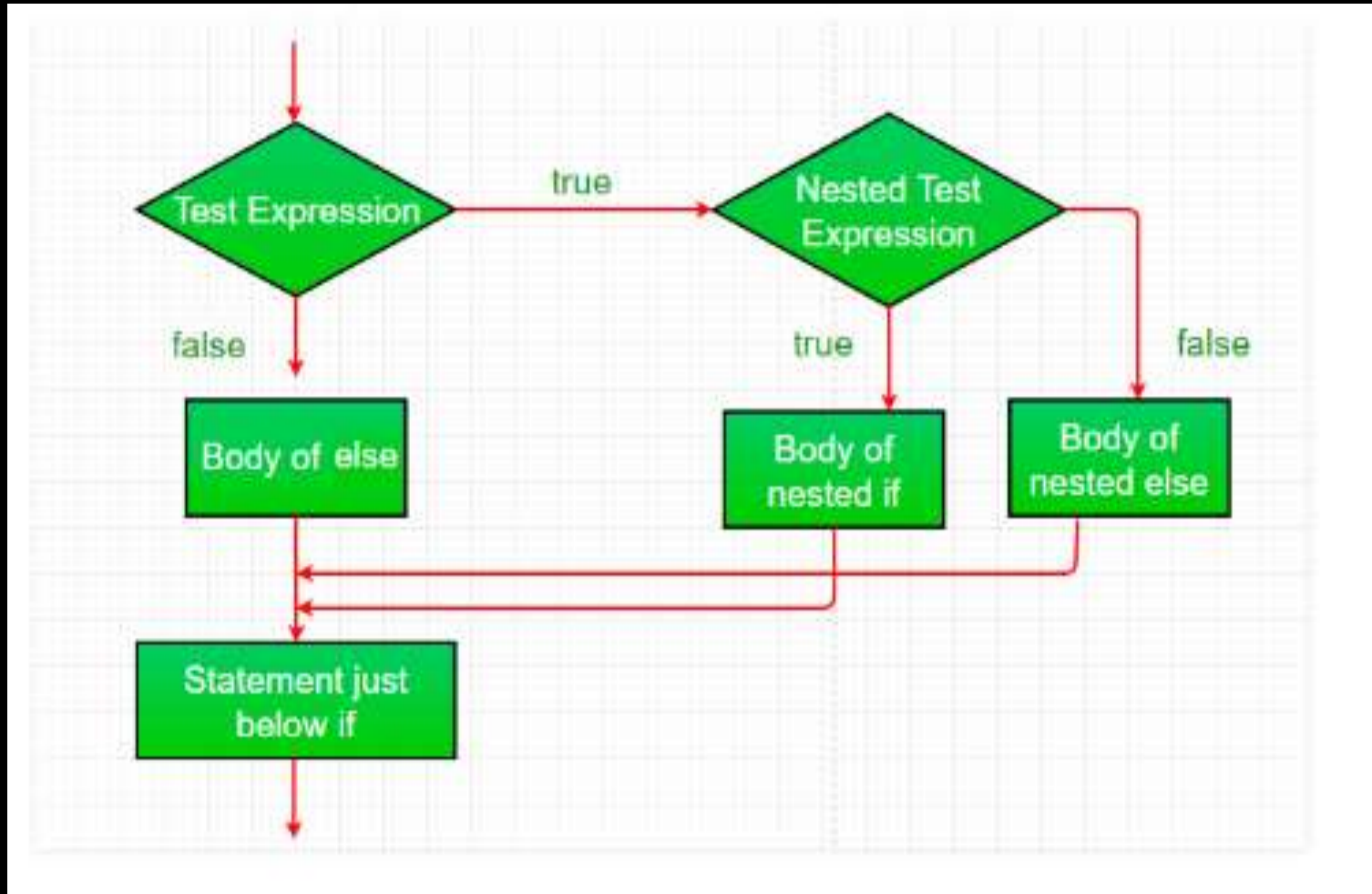
```
int choice = 0;

if (choice == 1)
{
    num = 1;
    num2 = 10;
}
else
{
    num = 2;
    num2 = 20;
}
```

Exercises:

1. Test if the value of the variable number is an odd number. If it is, print “This number is not an even number” otherwise, print “This number is an odd number.”

nested if structure



nested if structure

- A nested if is an if statement that is the target of another if or else. Nested if statements means an if statement inside an if statement. Yes, C language allows us to nest if statements within if statements. i.e, we can place an if statement inside another if statement.

```
if (condition1)
{
    // Executes when condition1 is true
    if (condition2)
    {
        // Executes when condition2 is true
    }
}
```

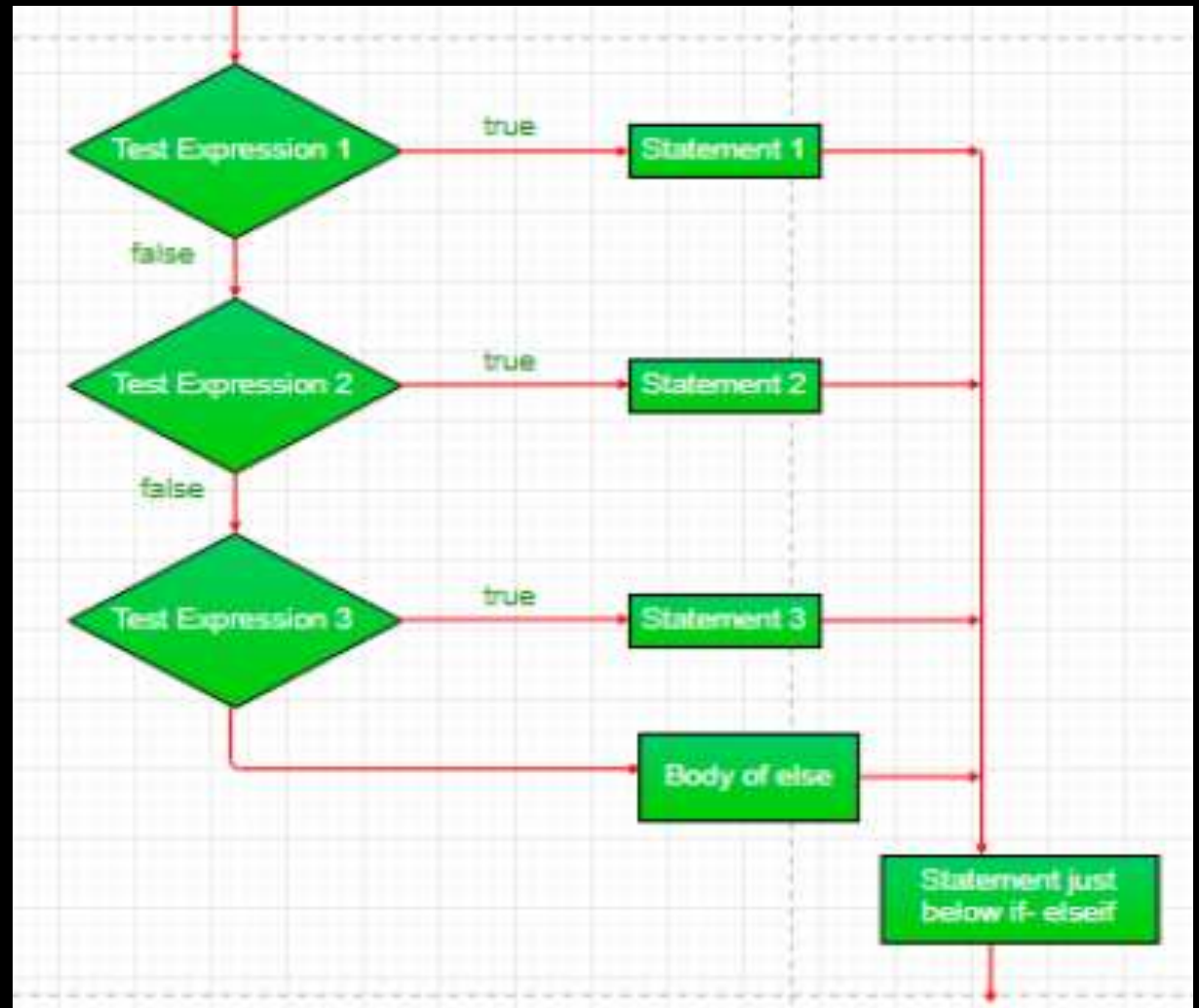
nested if structure

```
#include<stdio.h>
Int main()
{
    int i = 10;

    if (i == 10)
    {
        // First if statement
        if (i < 15)
            printf ("i is smaller than 15");
        // Nested - if statement
        // Will only be executed if statement above
        // it is true
        if (i < 12)
            printf("i is smaller than 12 too");
        else
            printf ("i is greater than 15");
    }
}
```

if else- if structure

- user can decide among multiple options. The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.



if

else- if structure

```
if (condition)
    statement;
else if (condition)
    statement; . .
else statement;
```


if

else- if structure

```
#include<stdio.h>

Int main()
{
    int i = 20;

    if (i == 10)
        printf("i is 10");
    else if (i == 15)
        printf("i is 15");
    else if (i == 20)
        printf("i is 20");
    else
        printf ("i is not present");
}
}
```